



University of Pisa

Department of Computer Science  
Bachelor's Degree

Bachelor's thesis

# A FRI-Based Polynomial Commitment Scheme

Supervisors:

**Prof. Anna Bernasconi**  
University of Pisa

**Dott. Giorgio dell'Immagine**  
\*\*non lo so\*\*

Candidate:

**Sofia Scalzo**  
University of Pisa

Academic year 2025/2026

# Abstract

# Contents

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>Abstract</b>                                               | <b>2</b>  |
| <b>1 Mathematical and Cryptographic Background</b>            | <b>5</b>  |
| 1.1 Finite Fields . . . . .                                   | 5         |
| 1.1.1 The BabyBear Field . . . . .                            | 5         |
| 1.1.2 Roots of Unity . . . . .                                | 5         |
| 1.1.3 Field Extensions . . . . .                              | 6         |
| 1.2 Polynomials . . . . .                                     | 6         |
| 1.2.1 The Number Theoretic Transform . . . . .                | 6         |
| 1.2.2 Low-Degree Extension . . . . .                          | 7         |
| 1.2.3 Vanishing Polynomials . . . . .                         | 7         |
| 1.3 Coding Theory . . . . .                                   | 7         |
| 1.4 Reed-Solomon Codes . . . . .                              | 8         |
| 1.4.1 Relative Hamming Distance . . . . .                     | 8         |
| 1.5 Cryptographic Hash Functions . . . . .                    | 9         |
| 1.6 Proof Systems . . . . .                                   | 9         |
| 1.6.1 Interactive Proofs . . . . .                            | 9         |
| 1.6.2 Interactive Arguments . . . . .                         | 10        |
| 1.6.3 The Fiat-Shamir Transformation . . . . .                | 10        |
| 1.7 Polynomial Commitment Schemes . . . . .                   | 10        |
| <b>2 Interactive Oracle Proofs and the BCS Transformation</b> | <b>12</b> |
| 2.1 Merkle Trees . . . . .                                    | 12        |
| 2.2 Interactive Oracle Proofs . . . . .                       | 13        |
| 2.3 The BCS Transformation . . . . .                          | 14        |
| <b>3 The FRI Protocol</b>                                     | <b>15</b> |
| 3.1 Overview and Problem Statement . . . . .                  | 15        |
| 3.2 The Evaluation Domain . . . . .                           | 16        |
| 3.2.1 Requirements . . . . .                                  | 16        |
| 3.2.2 Construction . . . . .                                  | 16        |
| 3.3 The Folding Operation . . . . .                           | 18        |
| 3.3.1 Even-Odd Decomposition . . . . .                        | 18        |
| 3.3.2 Recovering the Components from a Pair . . . . .         | 18        |
| 3.3.3 The Fold . . . . .                                      | 19        |
| 3.4 The Protocol . . . . .                                    | 20        |
| 3.4.1 Commit Phase . . . . .                                  | 20        |

|                     |                                                  |           |
|---------------------|--------------------------------------------------|-----------|
| 3.4.2               | Query Phase . . . . .                            | 21        |
| 3.4.3               | Verification and the Colinearity Check . . . . . | 21        |
| 3.5                 | Soundness . . . . .                              | 22        |
| 3.5.1               | The Two Things That Can Go Wrong . . . . .       | 22        |
| 3.5.2               | Why Folding Preserves Distance . . . . .         | 22        |
| 3.5.3               | The Soundness Theorem . . . . .                  | 23        |
| 3.5.4               | Beyond the Provable Regime . . . . .             | 24        |
| <b>Bibliography</b> |                                                  | <b>25</b> |

# Chapter 1

## Mathematical and Cryptographic Background

This chapter introduces the notation and mathematical foundations used throughout the thesis.

### 1.1 Finite Fields

**Definition 1.1** (Prime Field). Let  $p$  be a prime number. The *prime field*  $\mathbb{F}_p$  is the set  $\{0, 1, \dots, p-1\}$  equipped with addition and multiplication modulo  $p$ .

The multiplicative group  $\mathbb{F}_p^* = \mathbb{F}_p \setminus \{0\}$  is cyclic of order  $p-1$ . An element  $g \in \mathbb{F}_p^*$  is called a *generator* if  $\{g^0, g^1, \dots, g^{p-2}\} = \mathbb{F}_p^*$ . For any  $a \in \mathbb{F}_p^*$ , the multiplicative inverse can be computed via Fermat's little theorem:

$$a^{-1} \equiv a^{p-2} \pmod{p}. \quad (1.1)$$

**Example 1.1.** In  $\mathbb{F}_7 = \{0, 1, \dots, 6\}$  arithmetic wraps around modulo 7: for instance  $5 + 4 = 2$  and  $5 \cdot 4 = 6$ . The inverse of 3 is  $3^{p-2} = 3^5 = 5$ , since  $3 \cdot 5 = 15 = 1 \pmod{7}$ .

#### 1.1.1 The BabyBear Field

The implementation described in this thesis operates over the BabyBear prime field, defined by

$$p = 2^{31} - 2^{27} + 1 = 2013\,265\,921. \quad (1.2)$$

Since  $p < 2^{31}$ , every field element fits in a 32-bit unsigned integer, and the product of two elements fits in a 64-bit integer.

The multiplicative group  $\mathbb{F}_p^*$  has order  $p-1 = 2^{27} \cdot 15$ . The factor  $2^{27}$ , the *two-adicity* of the field, means that  $\mathbb{F}_p^*$  contains a cyclic subgroup of order  $2^{27}$ , which is essential for the Number Theoretic Transform.

#### 1.1.2 Roots of Unity

**Definition 1.2** (Primitive Root of Unity). An element  $\omega \in \mathbb{F}_p$  is a *primitive  $N$ -th root of unity* if  $\omega^N = 1$  and  $\omega^k \neq 1$  for all  $0 < k < N$ .

**Example 1.2.** In  $\mathbb{F}_7$  the element  $\omega = 2$  is a primitive 3rd root of unity:  $2^1 = 2$ ,  $2^2 = 4$ ,  $2^3 = 1$ , and no smaller power gives 1. Its powers form the subgroup  $\{1, 2, 4\}$  of order 3.

Given a primitive  $N$ -th root of unity  $\omega$ , the set  $H = \{1, \omega, \omega^2, \dots, \omega^{N-1}\}$  is a multiplicative subgroup of  $\mathbb{F}_p^*$  of order  $N$ . Because BabyBear has two-adicity 27, subgroups of order  $2^k$  for any  $k \leq 27$  exist and are generated from a fixed multiplicative generator  $g$  by taking  $g^{(p-1)/2^k}$ .

These subgroups serve as evaluation domains for the polynomials in the Scalable Transparent Argument of Knowledge (STARK) protocol: the execution trace is evaluated on a subgroup  $H$  of order  $2^h$ , and the Fast Reed-Solomon Interactive Oracle Proof of Proximity (FRI) operates on a larger domain.

### 1.1.3 Field Extensions

**Definition 1.3** (Field Extension). A *field extension*  $\mathbb{F}_{p^k}$  of degree  $k$  over  $\mathbb{F}_p$  is the quotient ring  $\mathbb{F}_p[X]/(f(X))$ , where  $f(X) \in \mathbb{F}_p[X]$  is an irreducible polynomial of degree  $k$ . Elements are polynomials of degree less than  $k$  with coefficients in  $\mathbb{F}_p$ .

In this work we use the *quartic extension*  $\mathbb{F}_{p^4}$ , constructed via the irreducible polynomial  $X^4 - 11$ . An element of  $\mathbb{F}_{p^4}$  is a tuple  $(c_0, c_1, c_2, c_3) \in \mathbb{F}_p^4$  representing  $c_0 + c_1X + c_2X^2 + c_3X^3$ , with the reduction rule  $X^4 = 11$ . The extension has  $p^4 \approx 2^{124}$  elements, large enough that a folding challenge sampled from it makes the soundness error negligible, as discussed in Chapter 3.

**Example 1.3.** A smaller instance is  $\mathbb{F}_{49} = \mathbb{F}_7[X]/(X^2 + 1)$ , which is a field because  $X^2 + 1$  has no root in  $\mathbb{F}_7$ . Its elements are  $a + bX$  with  $a, b \in \mathbb{F}_7$  and the rule  $X^2 = -1$ . For example,  $(2 + 3X)(1 + X) = 2 + 5X + 3X^2 = 2 + 5X - 3 = 6 + 5X$ .

## 1.2 Polynomials

A univariate polynomial over  $\mathbb{F}_p$  of degree at most  $d$  is  $f(X) = \sum_{i=0}^d c_i X^i$  with  $c_i \in \mathbb{F}_p$ . It can be represented by its *coefficient vector*  $(c_0, \dots, c_d)$  or by its *evaluation vector* on a domain  $D$ . Converting between the two representations, and evaluating a polynomial at a point, are the fundamental operations of the prover.

### 1.2.1 The Number Theoretic Transform

The conversion between coefficient and evaluation representations is performed using the Number Theoretic Transform (NTT), the finite-field analogue of the Fast Fourier Transform. When the evaluation domain is a multiplicative subgroup  $D = \{1, \omega, \dots, \omega^{N-1}\}$  with  $N$  a power of 2, the NTT computes all evaluations  $f(\omega^i)$  in  $O(N \log N)$  operations. The algorithm exploits the decomposition

$$f(X) = f_{\text{even}}(X^2) + X \cdot f_{\text{odd}}(X^2),$$

recursing on the even- and odd-indexed coefficients over the squared subgroup, which has half the size. The inverse transform runs the same routine with  $\omega^{-1}$  and divides by  $N$ .

### 1.2.2 Low-Degree Extension

The *low-degree extension* (LDE) of a polynomial is its evaluation on a domain larger than its coefficient count. Given the  $n$  coefficients of  $f$ , the LDE with blowup factor  $b$  evaluates  $f$  on a coset of a subgroup of size  $b \cdot n$ . The resulting evaluation vector is the Reed-Solomon codeword that the prover commits to. In our implementation the low-degree extension is computed by padding the coefficients with zeros, applying a coset shift, and running the NTT on the enlarged domain [Sta21, Section 3.4].

### 1.2.3 Vanishing Polynomials

**Definition 1.4** (Vanishing Polynomial [Sta21, Section 5.1]). Given a set  $S \subseteq \mathbb{F}$ , the *vanishing polynomial* of  $S$  is  $Z_S(X) := \prod_{s \in S} (X - s)$ . When  $S$  is a multiplicative subgroup  $H = \{1, \omega, \dots, \omega^{N-1}\}$ , this simplifies to  $Z_H(X) = X^N - 1$ .

**Example 1.4.** Take the subgroup  $H = \{1, 2, 4\}$  of  $\mathbb{F}_7^*$  from Example 1.2. Its vanishing polynomial is  $Z_H(X) = (X - 1)(X - 2)(X - 4) = X^3 - 1$ , which evaluates to 0 at each of 1, 2, 4 and is nonzero elsewhere, for instance  $Z_H(3) = 5$ .

$Z_H$  vanishes on every element of  $H$  and nowhere else. If a constraint polynomial  $c(X)$  must be zero on  $H$ , then  $Z_H(X)$  divides  $c(X)$ , and the quotient  $q(X) = c(X)/Z_H(X)$  is a polynomial. The verifier checks that  $q$  has low degree, which certifies that  $c$  vanishes on  $H$  [Sta21]. This mechanism is the basis of the STARK construction.

## 1.3 Coding Theory

We recall the standard notions from coding theory that the soundness analysis of the FRI protocol relies on. The specific family of codes used in this work, the Reed-Solomon codes, is introduced in Section 1.4 as an instance of these general definitions.

**Definition 1.5** (Error-Correcting Code [Arn+24, Definition 3.1]). An *error-correcting code* of length  $n$  over an alphabet  $\Sigma$  is a subset  $C \subseteq \Sigma^n$ . The code is a *linear code* if  $\Sigma = \mathbb{F}$  is a field and  $C$  is a linear subspace of  $\mathbb{F}^n$ . The *rate* of a linear code is  $\rho := \dim(C)/n$ .

**Example 1.5.** The set of all vectors  $(a, b, a + b) \in \mathbb{F}_7^3$  is a linear code of length  $n = 3$ : it is a subspace of dimension 2 (the free choices  $a$  and  $b$ ), so its rate is  $\rho = 2/3$ . The third coordinate is a built-in check that lets some errors be detected.

The elements of  $C$  are its *codewords*, and the *minimum distance* of a code is the smallest relative Hamming distance (Definition 1.8) between two distinct codewords. When a word lies far from every codeword, several codewords may still lie within a given distance of it; the following notion quantifies this.

**Definition 1.6** (List Decoding [Arn+24, Definition 3.2]). For a code  $C \subseteq \Sigma^n$ , a word  $f \in \Sigma^n$ , and a proximity parameter  $\delta \in [0, 1]$ , the *list* of  $C$  at  $f$  within radius  $\delta$  is

$$\Lambda(C, f, \delta) := \{u \in C : \Delta(f, u) \leq \delta\}.$$

The code is  $(\delta, \ell)$ -*list decodable* if  $|\Lambda(C, f, \delta)| \leq \ell$  for every  $f \in \Sigma^n$ . A radius  $\delta$  is *within unique decoding* if the code is  $(\delta, 1)$ -list decodable.

**Example 1.6.** Within the unique decoding radius there is at most one codeword near a received word, so the list  $\Lambda(C, f, \delta)$  has size at most 1 and decoding is unambiguous. Past that radius the list can grow: a word sitting exactly between two codewords has both of them in its list, so the code is no longer  $(\delta, 1)$ -list decodable there.

For a code of minimum distance  $\delta_C$ , any radius  $\delta \leq \delta_C/2$  is within unique decoding. Beyond it, the list size is controlled, for the codes we use, by the Johnson bound stated below.

**Theorem 1.1** (Johnson Bound [Arn+24, Theorem 4.3]). *The Reed-Solomon code of rate  $\rho$  is  $(1 - \sqrt{\rho} - \eta, \frac{1}{2\eta\sqrt{\rho}})$ -list decodable for every  $\eta \in (0, 1 - \sqrt{\rho})$ .*

**Example 1.7.** For rate  $\rho = 1/2$ , the unique decoding radius is  $\frac{1-\rho}{2} = 0.25$ , while the Johnson radius is  $1 - \sqrt{1/2} \approx 0.29$ . Between these two values a received word may have a few codewords in its list, but the Johnson bound guarantees the number stays small, which is what the soundness analysis needs.

The two thresholds, the *unique decoding* radius  $\frac{1-\rho}{2}$  and the *Johnson* radius  $1 - \sqrt{\rho}$ , are exactly where the soundness guarantees of FRI change form (Chapter 3).

## 1.4 Reed-Solomon Codes

**Definition 1.7** (Reed-Solomon Code [BS+18, Section 2.1]). For a finite field  $\mathbb{F}$ , a subset  $S \subseteq \mathbb{F}$ , and a rate parameter  $\rho \in (0, 1)$ , the *Reed-Solomon code*  $\text{RS}[\mathbb{F}, S, \rho]$  is the set of functions  $f : S \rightarrow \mathbb{F}$  that are evaluations of polynomials of degree less than  $\rho|S|$ :

$$\text{RS}[\mathbb{F}, S, \rho] = \{(f(x))_{x \in S} : f \in \mathbb{F}[X], \deg(f) < \rho|S|\}.$$

**Example 1.8.** Take  $\mathbb{F}_7$ , evaluation set  $S = \{1, 2, 3, 4, 5, 6\}$ , and rate  $\rho = 1/3$ , so codewords are evaluations of polynomials of degree less than  $\rho|S| = 2$ , that is, lines  $a + bX$ . The polynomial  $f(X) = 2 + 3X$  gives the codeword  $(f(1), \dots, f(6)) = (5, 1, 4, 0, 3, 6)$ . Any two distinct lines agree in at most one point, so their codewords differ in at least five of the six positions.

Each codeword is the evaluation of a polynomial of degree less than  $\rho|S|$  on all points of  $S$ . The code has minimum distance  $|S| - \rho|S| + 1$ , so two distinct codewords differ on a large fraction of positions.

The parameter  $\rho$  is the *rate*, and  $1/\rho$  is the *blowup factor*. In our implementation the blowup factor is 2 ( $\rho = 1/2$ ): the codeword produced by the low-degree extension is twice the length of the coefficient vector.

### 1.4.1 Relative Hamming Distance

**Definition 1.8** (Relative Hamming Distance). The *relative Hamming distance* between two vectors  $u, v \in \mathbb{F}^n$  is

$$\Delta(u, v) = \frac{|\{i : u_i \neq v_i\}|}{n}.$$

The *distance* of a vector  $u$  from a code  $C$  is  $\Delta(u, C) = \min_{v \in C} \Delta(u, v)$ . A vector  $u$  is  $\delta$ -close to  $C$  if  $\Delta(u, C) \leq \delta$ , and  $\delta$ -far if  $\Delta(u, C) > \delta$ .



**Example 1.9.** The codeword  $u = (5, 1, 4, 0, 3, 6)$  of Example 1.8 and the corrupted word  $v = (5, 1, 4, 2, 3, 6)$  differ in exactly one of six positions, so  $\Delta(u, v) = 1/6$ . If every genuine codeword differs from  $v$  in at least this many positions, then  $v$  is  $\frac{1}{6}$ -far from the code.

The FRI protocol tests proximity to a Reed-Solomon codeword. As stated in [BS+18, Section 2.1], the verifier’s task is to distinguish between the case that  $f \in \text{RS}[\mathbb{F}, S, \rho]$  and the case that  $f$  is far from  $\text{RS}[\mathbb{F}, S, \rho]$  in relative Hamming distance.

## 1.5 Cryptographic Hash Functions

**Definition 1.9** (Collision-Resistant Hash Function). A *collision-resistant hash function* is a function  $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$  such that no efficient algorithm can find distinct inputs  $x \neq x'$  with  $H(x) = H(x')$ , except with negligible probability.

**Example 1.10.** SHA-256 maps any input to a 256-bit digest. Changing even one bit of the input produces an essentially unrelated digest, and no method is known to find two inputs with the same digest faster than the roughly  $2^{128}$  tries of a birthday attack, which is what collision resistance requires.

Hash functions serve two roles in STARK systems: constructing Merkle trees for committing to polynomial evaluations, and deriving the verifier’s challenges via the Fiat-Shamir heuristic. Our implementation uses two different hash functions for these two roles: SHA-256 for the Merkle commitments and Keccak- $f[1600]$  for the challenger.

The distinction matters for the Fiat-Shamir transformation. Chiesa and Orrù [CO25] prove that a *duplex sponge* construction (such as Keccak/SHA-3) is suitable for Fiat-Shamir, while Merkle-Damgård constructions (such as SHA-256) lack a comparable security proof in this setting. For this reason we use a Keccak-based sponge for the challenger, while retaining SHA-256 for the Merkle trees, where only collision resistance is required.

## 1.6 Proof Systems

We now establish the formal framework of proof systems, following Thaler [Tha22].

### 1.6.1 Interactive Proofs

**Definition 1.10** (Interactive Proof [Tha22, Definition 3.1]). An *interactive proof system* for a language  $\mathcal{L}$  is a pair  $(\mathcal{P}, \mathcal{V})$  where  $\mathcal{P}$  is a computationally unbounded prover and  $\mathcal{V}$  is a probabilistic polynomial-time verifier. The protocol proceeds in rounds, and must satisfy:

- **Completeness.** For every  $x \in \mathcal{L}$ ,  $\Pr[\mathcal{V} \text{ accepts}] \geq 1 - \delta_c$ .
- **Soundness.** For every  $x \notin \mathcal{L}$  and every cheating prover  $\mathcal{P}^*$ ,  $\Pr[\mathcal{V} \text{ accepts}] \leq \delta_s$ .

**Example 1.11.** A classic interactive proof is for graph non-isomorphism. The verifier secretly picks one of two graphs, randomly relabels its vertices, and sends the result to the prover, who must say which graph it came from. If the graphs are non-isomorphic an unbounded prover always answers correctly; if they are isomorphic no prover can do better than guessing, so the verifier catches a false claim with probability  $1/2$  per round.

## 1.6.2 Interactive Arguments

Relaxing the soundness of an interactive proof to hold only against computationally efficient provers yields a weaker but more flexible notion.

**Definition 1.11** (Interactive Argument [Tha22, Section 3.2]). An *interactive argument* for a language  $\mathcal{L}$  is an interactive proof in which the soundness condition is only required to hold against prover strategies that run in probabilistic polynomial time.

**Example 1.12.** The FRI-based systems in this thesis are arguments, not proofs. Their soundness rests on the collision resistance of a hash function: a prover with unlimited time could find hash collisions and forge a convincing transcript, but a realistic polynomial-time prover cannot, so soundness holds against every efficient prover.

Both models share the same structure. In a *public-coin* protocol, every verifier message is a uniformly random challenge, independent of the prover’s messages, and the verifier’s decision is a deterministic function of the *transcript*, the ordered sequence of messages and challenges. These are exactly the protocols to which the Fiat-Shamir transformation (Subsection 1.6.3) applies. An *argument of knowledge* further requires an efficient extractor  $\mathcal{E}$  that recovers a valid witness from any convincing prover.

STARKs are *arguments*, not *proofs* in the formal sense: their soundness relies on the collision resistance of a hash function, which a computationally unbounded prover could break. The interactive oracle proofs of Section 2.2 specialise this model by providing the prover’s messages as oracles.

## 1.6.3 The Fiat-Shamir Transformation

A *non-interactive* proof consists of a single message from the prover to the verifier. The Fiat-Shamir transformation removes interaction from a public-coin protocol by replacing the verifier’s random challenges with the output of a hash function applied to the transcript so far. Under the random oracle model, this preserves soundness. Chiesa and Orrù [CO25] give a formal treatment using a duplex sponge, which resolves the ambiguity between writing to and reading from the transcript by using the sponge’s native *absorb* and *squeeze* operations. Our challenger implements this construction over Keccak- $f$ [1600].

## 1.7 Polynomial Commitment Schemes

**Definition 1.12** (Polynomial Commitment Scheme). A *Polynomial Commitment Scheme* (PCS) consists of four algorithms:

- $\text{Setup}(\lambda, d) \rightarrow \text{pp}$ : generates public parameters for polynomials of degree at most  $d$ .
- $\text{Commit}(\text{pp}, f) \rightarrow C$ : produces a commitment  $C$  to the polynomial  $f$ .
- $\text{Open}(\text{pp}, f, z) \rightarrow (v, \pi)$ : produces a proof  $\pi$  that  $f(z) = v$ .
- $\text{Verify}(\text{pp}, C, z, v, \pi) \rightarrow \{0, 1\}$ : checks the evaluation proof.

The scheme must satisfy correctness and binding.

**Example 1.13.** A prover commits to a polynomial  $f$  by sending a single value  $C$ . Later, asked for the value at a point  $z$ , it returns  $v = f(z)$  with a proof  $\pi$ . The verifier checks  $\pi$  against  $C$ ,  $z$ , and  $v$  without ever seeing  $f$  itself; binding ensures the prover cannot open the same  $C$  to two different values at the same  $z$ .

The STARK system implemented in this thesis uses a *FRI-based PCS* [BS+18]: a polynomial is committed via a Merkle tree over its low-degree extension, and evaluation proofs are constructed through the FRI low-degree testing protocol. This approach requires no trusted setup (transparent) and relies only on hash functions (post-quantum secure). The FRI protocol is described in detail in Chapter 3.

# Chapter 2

## Interactive Oracle Proofs and the BCS Transformation

The FRI protocol of Chapter 3 is naturally expressed as an *interactive oracle proof* (IOP): the prover sends oracles that the verifier queries at a few positions, rather than messages it reads in full. This chapter introduces the IOP model and the BCS transformation that compiles an IOP into a non-interactive argument by realising its oracles as Merkle commitments and its queries as commitment openings. Once this compiler is in place, the FRI protocol can be described purely in terms of oracles and queries, leaving the commitment machinery implicit.

### 2.1 Merkle Trees

**Definition 2.1** (Merkle Tree). A *Merkle tree* over a sequence  $(v_0, v_1, \dots, v_{N-1})$  with  $N = 2^k$  is a binary tree where each leaf stores the hash of one element,  $\ell_i = H(v_i)$ ; each internal node stores the hash of the concatenation of its two children,  $H(n_{\text{left}} \| n_{\text{right}})$ ; and the root  $r$  is a single digest that binds the entire sequence.

**Example 2.1.** For four values  $(v_0, v_1, v_2, v_3)$  the tree has leaves  $\ell_i = H(v_i)$ , two internal nodes  $H(\ell_0 \| \ell_1)$  and  $H(\ell_2 \| \ell_3)$ , and a root hashing the two. To prove  $v_2$  sits at position 2, the prover reveals  $\ell_3$  and  $H(\ell_0 \| \ell_1)$ ; the verifier recomputes the root from  $v_2$  and these two siblings and checks it against the committed root (Figure 2.1).

To prove that  $v_i$  is the value at position  $i$ , the prover provides an *authentication path*: the  $\log_2(N)$  sibling hashes along the path from the leaf to the root. The verifier recomputes the root and accepts if it matches  $r$ . Verification requires  $O(\log N)$  hash evaluations.

In the BCS (Ben-Sasson–Chiesa–Spooner) transformation [BSCS16], Merkle trees commit to the oracle messages of the interactive oracle proof (IOP, Section 2.2): the prover evaluates a polynomial on the FRI domain, builds a Merkle tree over the evaluations, and sends the root to the verifier.

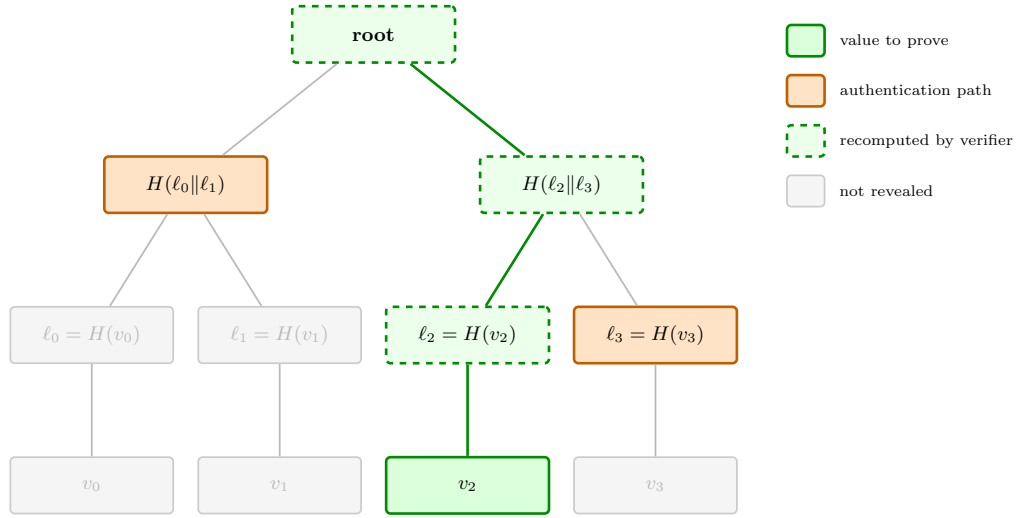


Figure 2.1: Verification of  $v_2$  in a four-leaf Merkle tree. The prover reveals the *authentication path* (the two siblings  $\ell_3$  and  $H(\ell_0 \parallel \ell_1)$ , in orange); the verifier recomputes the dashed nodes from  $v_2$  up to the root and compares the result with the committed root. The greyed-out nodes are never revealed.

## 2.2 Interactive Oracle Proofs

**Definition 2.2** (Interactive Oracle Proof [BSCS16, Definition 2.2]). An *Interactive Oracle Proof* (IOP) for a language  $\mathcal{L}$  is an interactive protocol between a prover  $\mathcal{P}$  and a verifier  $\mathcal{V}$  that proceeds in rounds. In each round:

1. the prover sends an *oracle message*  $\pi_i$  (a function that the verifier may query at chosen positions);
2. the verifier sends a random challenge  $r_i$ .

After  $k$  rounds, the verifier makes a bounded number of queries to  $\pi_1, \dots, \pi_k$  and outputs *accept* or *reject*.

**Example 2.2.** FRI is itself an IOP. In each round the prover's oracle is a codeword, and the verifier's challenge is a folding coefficient; at the end the verifier queries only a handful of positions of each oracle rather than reading the full codewords, which is what makes verification succinct. The oracles are treated abstractly here; their concrete realisation as Merkle commitments is deferred to the BCS transformation.

The key difference from a standard interactive proof is that the verifier has *oracle access* to the prover's messages: rather than receiving each message in full, it may query a small number of chosen positions of  $\pi_i$ . The oracle is an abstraction: it fixes the message in advance, so that every query is answered consistently, but says nothing about how this is realised. Making the oracles concrete is the job of the BCS transformation of Section 2.3, after which we can reason about FRI purely in terms of oracles and queries, without committing to any particular implementation of them.

An IOP of *proximity* (IOPP) is an IOP in which the verifier tests whether the prover’s initial oracle is close to a codeword of a specified code, rather than testing membership in a language [BS+18]. FRI, described in Chapter 3, is an IOPP for Reed-Solomon codes.

## 2.3 The BCS Transformation

The IOP model of Section 2.2 leaves two things abstract: the oracle messages, which the verifier can query but not read in full, and the interaction, which requires the verifier to send live random challenges. The BCS transformation [BSCS16] removes both, compiling any public-coin IOP into a non-interactive argument in the random oracle model. It does so by realising each abstract ingredient with a concrete cryptographic one.

- **Oracles become commitments.** Each oracle message  $\pi_i$  is replaced by a *vector commitment* to its values: the prover builds a Merkle tree (Section 2.1) over  $\pi_i$  and sends only the root. The binding property of the commitment plays the role the oracle played in the IOP, fixing the message so that it cannot be changed after the fact.
- **Queries become openings.** When the verifier queries position  $j$  of  $\pi_i$ , the prover answers with the value together with its Merkle authentication path, and the verifier checks the path against the committed root. An opening that verifies is exactly a consistent answer to an oracle query.
- **Challenges become Fiat-Shamir outputs.** Each verifier challenge is derived by hashing the transcript so far (Subsection 1.6.3), so no live interaction is needed and the whole exchange becomes a single message.

The result is a non-interactive argument of knowledge in the random oracle model. Because of this compiler, once an IOP is specified we may describe it purely at the level of oracles and queries: it is enough to say that the prover sends the oracle for a purported codeword and that the verifier queries it at chosen positions, with the understanding that the oracles are Merkle vector commitments and the queries their openings. We adopt exactly this style in Chapter 3, and note there that the resulting FRI IOP is compiled into a non-interactive argument through the BCS transformation described here.

# Chapter 3

## The FRI Protocol

This chapter presents the Fast Reed-Solomon Interactive Oracle Proof of Proximity, the protocol at the core of the system implemented in this thesis. We begin by stating the problem FRI solves, then describe the algebraic structure of the evaluation domain, and finally the folding operation that drives the protocol.

### 3.1 Overview and Problem Statement

Recall from Definition 1.7 that, for a subset  $S \subseteq \mathbb{F}$  and a rate parameter  $\rho \in (0, 1)$ , the Reed-Solomon code  $\text{RS}[\mathbb{F}, S, \rho]$  consists of the evaluations on  $S$  of polynomials of degree less than  $\rho|S|$ . The problem FRI addresses is the following.

**Definition 3.1** (Rate Proximity Testing (RPT) [BS+18, Section 2.1]). A verifier is given oracle access to a function  $f : S \rightarrow \mathbb{F}$ , together with auxiliary information supplied by a prover. The verifier must distinguish, with high probability and using only a small number of queries, between the case that  $f \in \text{RS}[\mathbb{F}, S, \rho]$  and the case that  $f$  is far from every member of  $\text{RS}[\mathbb{F}, S, \rho]$  in relative Hamming distance (Definition 1.8).

RPT is a special case of the low-degree testing problem, and Ben-Sasson et al. identify it as a major bottleneck for transparent proof systems [BS+18, Section 2.1].

A naive solution is instructive. The verifier could ask the prover to open  $f$  at  $\rho|S|$  positions, interpolate the unique polynomial of degree less than  $\rho|S|$  through them, and check that it agrees with  $f$  elsewhere. This fails on two counts: the number of openings is proportional to the degree bound, which can reach  $2^{20}$  or more in practical instances, and every opening carries a Merkle authentication path of  $\log |S|$  hash values (Section 2.1), so the proof size becomes prohibitive.

FRI takes a different route. Instead of testing the degree directly, it *reduces* it: the prover repeatedly halves the degree of the committed polynomial using randomness supplied by the verifier, until the degree is small enough to send the remaining polynomial directly. Each reduction step is committed, and the verifier spot-checks the consistency of the steps at a small number of random positions. If the original function is far from the code, then with high probability at least one reduction is inconsistent, and the verifier detects it.

This yields a protocol with prover arithmetic complexity strictly linear in  $|S|$  and verifier arithmetic complexity strictly logarithmic [BS+18, Section 2.1]. In the ter-

minology of Section 2.2, FRI is an interactive oracle proof of proximity (IOPP) for Reed-Solomon codes: the prover’s messages are committed codewords which the verifier queries at a bounded number of positions, rather than reading in full.

*Remark 3.1.* FRI establishes *proximity*, not membership. A successful verification does not certify that the committed vector is exactly a Reed-Solomon codeword, only that it is close to one in relative Hamming distance. This distinction is central to the soundness analysis of the FRI protocol.

## 3.2 The Evaluation Domain

The folding step of FRI relies on specific algebraic structure in the evaluation domain. We state the requirements first, then give the construction used in our implementation.

### 3.2.1 Requirements

The reduction from a polynomial  $f$  to a polynomial of half the degree is carried out by pairing the evaluations of  $f$  at  $x$  and at  $-x$ . For this to be possible at every position, the domain must be closed under negation. Moreover, the folded polynomial is committed not on  $D$  but on the squared domain  $D^{(2)} := \{x^2 : x \in D\}$ , and since the folding step is applied iteratively,  $D^{(2)}$  must be closed under negation as well, and so on for every subsequent layer [Sta21, Section 3.9.1].

We therefore require a chain of domains

$$D = D_0, \quad D_{i+1} = D_i^{(2)}, \quad i = 0, 1, \dots$$

such that every  $D_i$  is closed under negation and  $|D_{i+1}| = |D_i|/2$ . These requirements are met by taking  $D$  to be a coset of a multiplicative subgroup of order  $2^k$  [Sta21, Section 3.9.1].

A coset is used rather than the subgroup itself so that the evaluation domain is disjoint from the trace domain on which the polynomial being committed is defined [Sta21, Section 3.4]. Disjointness is required in the full STARK protocol, where constraint polynomials are expressed as rational functions whose denominators vanish on the trace domain; evaluating them on a disjoint domain keeps those denominators nonzero.

### 3.2.2 Construction

We build the domain from the roots of unity of Subsection 1.1.2. Let  $n$  be the number of coefficients of the polynomial to be committed,  $b$  the blowup factor, and  $N = b \cdot n = 2^k$  the size of the evaluation domain. Two elements are fixed:

$$\omega = \gamma^{(p-1)/2^k}, \quad g = \gamma^{(p-1)/2^{k+1}}, \quad (3.1)$$

where  $\gamma$  is a fixed multiplicative generator of  $\mathbb{F}_p^*$ . Here  $\omega$  is a primitive  $2^k$ -th root of unity: its powers  $\langle \omega \rangle = \{1, \omega, \dots, \omega^{N-1}\}$  form a cyclic group of  $N$  equally spaced points. The element  $g$  is a primitive  $2^{k+1}$ -th root of unity, one “half-step” finer, and



it plays the role of an *offset* that rotates the whole group. The two are linked by the invariant

$$g^2 = \omega, \quad (3.2)$$

which, as we show below, is exactly what makes the domain reproduce itself under squaring.

The evaluation domain is the coset

$$D = g \cdot \langle \omega \rangle = \{ g\omega^j : j = 0, \dots, N-1 \}, \quad (3.3)$$

i.e. the group  $\langle \omega \rangle$  shifted by the offset  $g$ . The codeword committed by the prover is  $(f(g\omega^j))_{j=0}^{N-1}$ , computed by scaling the  $i$ -th coefficient of  $f$  by  $g^i$  and applying the NTT (Subsections 1.2.1 and 1.2.2). In our implementation  $\omega$  and  $g$  are two-power roots of unity obtained from the fixed generator  $\gamma$ , and the codeword is obtained by the scaled NTT of Subsection 1.2.2.

Three properties follow, and together they justify the folding step.

**Proposition 3.1.** *With  $D$  as in Equation (3.3) and  $N = 2^k$ ,  $k \geq 1$ :*

1.  $\omega^{N/2} = -1$ ;
2.  $D$  is closed under negation:  $-g\omega^j = g\omega^{j+N/2}$  for  $j < N/2$ ;
3.  $D^{(2)} = \omega \cdot \langle \omega^2 \rangle$ , a coset of the subgroup of order  $N/2$ ; in particular  $|D^{(2)}| = N/2$ .

*Proof.* (1) The element  $\omega^{N/2}$  squares to  $\omega^N = 1$ , so it is a square root of unity. It cannot equal 1, since  $\omega$  has order exactly  $N$ ; as  $\mathbb{F}_p$  is a field, the only square roots of unity are  $\pm 1$ , hence  $\omega^{N/2} = -1$ .

(2) Using (1),  $g\omega^{j+N/2} = g\omega^j \cdot \omega^{N/2} = -g\omega^j$ , and the index  $j+N/2$  lies in  $\{0, \dots, N-1\}$ , so the negated element is again in  $D$ .

(3) Squaring an element of  $D$  and applying Equation (3.2) gives  $(g\omega^j)^2 = g^2\omega^{2j} = \omega^{2j+1}$ , so  $D^{(2)} = \{\omega^{2j+1} : j = 0, \dots, N-1\}$ , the subgroup  $\langle \omega^2 \rangle$  shifted by  $\omega$ . By (2) the map  $x \mapsto x^2$  identifies  $x$  with  $-x$ , so it is two-to-one on  $D$  and  $|D^{(2)}| = N/2$ .  $\square$

Property (3) is what makes the construction *self-reproducing*: the squared domain is again a coset of a two-power subgroup, of the same shape as  $D$  but half the size, now described by the pair  $(\omega, \omega^2)$  in place of  $(g, \omega)$ . A single FRI round therefore updates the domain simply by squaring both parameters,

$$g \leftarrow g^2, \quad \omega \leftarrow \omega^2, \quad (3.4)$$

that is, the two domain parameters are squared at the end of each layer. The requirements of Subsection 3.2.1 are preserved at every level, as long as the subgroup order remains even.

**Example 3.1.** Take  $\mathbb{F}_{17}$ , whose multiplicative group has order  $16 = 2^4$ , with generator  $\gamma = 3$ . For  $N = 8$  we get  $\omega = 3^2 = 9$  and  $g = 3^1 = 3$ , so  $g^2 = 9 = \omega$  as required by Equation (3.2). The evaluation domain is

$$D = \{3, 10, 5, 11, 14, 7, 12, 6\}.$$

Its elements pair up as  $(3, 14)$ ,  $(10, 7)$ ,  $(5, 12)$ ,  $(11, 6)$ , each pair summing to 0 mod 17, this is  $-g\omega^j = g\omega^{j+4}$  in action. Squaring gives  $D^{(2)} = \{9, 15, 8, 2\}$ , of size 4, which is exactly the coset  $\omega \cdot \langle \omega^2 \rangle = 9 \cdot \langle 13 \rangle$  built from the updated parameters.

### 3.3 The Folding Operation

Folding is the algebraic core of FRI. In one step it takes a polynomial  $f$  of degree less than  $d$ , given by its values on the domain  $D$ , and produces a new polynomial of degree less than  $d/2$ , given by its values on the smaller domain  $D^{(2)}$ . The reduction uses a single random challenge and no interpolation: the values of the new polynomial are a simple combination of the values of the old one. Repeating this step reduces the degree by half each time; the prover stops once the polynomial is small enough to send directly, and transmits its remaining coefficients in the clear.

The section is organised around the three questions that folding answers: how a polynomial splits into two halves (Subsection 3.3.1), how those halves are recovered from evaluations alone (Subsection 3.3.2), and how the challenge combines them into the folded polynomial (Subsection 3.3.3). The two polynomials  $f_e$  and  $f_o$  introduced next appear throughout the section in three guises. They are first defined as polynomials, then evaluated at a point, then recombined, but they are always the same two objects.

#### 3.3.1 Even-Odd Decomposition

The first step is to split  $f$  into the part carried by its even-indexed coefficients and the part carried by its odd-indexed ones. Grouping the coefficients this way lets us write  $f$  in terms of two polynomials in  $X^2$ , each of roughly half the degree.

**Definition 3.2** (Even-Odd Decomposition). Let  $f(X) = \sum_{i=0}^{d-1} c_i X^i \in \mathbb{F}_p[X]$ . Define

$$f_e(Y) = \sum_{i \text{ even}} c_i Y^{i/2}, \quad f_o(Y) = \sum_{i \text{ odd}} c_i Y^{(i-1)/2}.$$

Then  $f$  decomposes uniquely as

$$f(X) = f_e(X^2) + X \cdot f_o(X^2), \tag{3.5}$$

and both  $f_e$  and  $f_o$  have degree less than  $\lceil d/2 \rceil$ .

In words:  $f_e$  collects the even coefficients  $c_0, c_2, c_4, \dots$  and  $f_o$  the odd ones  $c_1, c_3, c_5, \dots$ , each repackaged as a polynomial in  $Y = X^2$ . Equation (3.5) just puts them back together, with the extra factor  $X$  accounting for the odd powers. Each half keeps at most  $d/2$  of the original  $d$  coefficients, which is why its degree is halved. This is the same split that drives the recursive step of the NTT (Subsection 1.2.1), and it is the decomposition FRI uses in the commit phase.

#### 3.3.2 Recovering the Components from a Pair

There is a catch: the prover does not hold  $f$  as a list of coefficients, but as a list of values on  $D$ . So the values of  $f_e$  and  $f_o$  must be recovered from the values of  $f$  alone. The trick is to evaluate  $f$  at a point  $x$  and at its negative  $-x$ . Because  $(-x)^2 = x^2$ , both evaluations land on the same point  $x^2$  of the squared domain, and the odd part flips sign:

$$\begin{aligned} f(x) &= f_e(x^2) + x \cdot f_o(x^2), \\ f(-x) &= f_e(x^2) - x \cdot f_o(x^2). \end{aligned} \tag{3.6}$$

These are two linear equations in the two unknowns  $f_e(x^2)$  and  $f_o(x^2)$ . Adding them isolates the even part; subtracting them isolates the odd part. Since  $x \neq 0$  for every  $x \in D$ , we can divide and obtain

$$f_e(x^2) = \frac{f(x) + f(-x)}{2}, \quad f_o(x^2) = \frac{f(x) - f(-x)}{2x}. \quad (3.7)$$

So a single pair of values  $(f(x), f(-x))$  determines both components at the point  $x^2 \in D^{(2)}$ . By Theorem 3.1 that pair is stored in the codeword at indices  $i$  and  $i + N/2$ , so the prover finds it by reading two entries that sit exactly half the list apart.

### 3.3.3 The Fold

The final step mixes the two halves using the verifier's random challenge  $\alpha$ . The mix collapses  $f_e$  and  $f_o$  into one polynomial of half the degree.

**Definition 3.3** (FRI Fold). Let  $f$  have degree less than  $d$  and let  $\alpha$  be a challenge sampled from a field  $\mathbb{K} \supseteq \mathbb{F}_p$ . The *fold* of  $f$  with respect to  $\alpha$  is

$$f^{(\alpha)}(Y) := f_e(Y) + \alpha \cdot f_o(Y), \quad (3.8)$$

a polynomial over  $\mathbb{K}$  of degree less than  $\lceil d/2 \rceil$ .

Putting Equation (3.8) together with Equation (3.7), the values of  $f^{(\alpha)}$  on  $D^{(2)}$  come straight from the values of  $f$  on  $D$ , with no interpolation anywhere:

$$f^{(\alpha)}(x^2) = \frac{f(x) + f(-x)}{2} + \alpha \cdot \frac{f(x) - f(-x)}{2x}. \quad (3.9)$$

This is exactly what our implementation does: for each  $i < N/2$  it reads the two entries at indices  $i$  and  $i + N/2$ , applies Equation (3.9) with  $x = g\omega^i$ , and writes out a codeword of length  $N/2$  on the domain  $D^{(2)}$  given by the updated parameters of Equation (3.4).

One round thus halves two things at once: the degree bound, by Definition 3.3, and the domain size, by Theorem 3.1. The rate  $\rho$  stays the same, so the folded codeword is again a Reed-Solomon codeword of the same rate, which is what lets the next round repeat the identical step. The prover keeps folding until the degree is small enough to send the remaining polynomial directly.

*Remark 3.2* (Arithmetic in the extension field). The initial codeword has entries in  $\mathbb{F}_p$ , but the challenge  $\alpha$  is sampled from the quartic extension  $\mathbb{F}_{p^4}$  (Subsection 1.1.3). Because Equation (3.8) multiplies  $f_o$  by  $\alpha$ , the folded polynomial already has coefficients in  $\mathbb{F}_{p^4}$ , and so does every layer after it. In our implementation this switch happens at the very first fold: the initial codeword holds base-field elements, every later layer holds extension-field elements, and the Merkle leaves grow accordingly from four to sixteen bytes.

**Example 3.2.** We continue Example 3.1. Let  $f(X) = 1 + 2X + 3X^2 + 4X^3$  over  $\mathbb{F}_{17}$ , so  $d = 4$ ,  $N = 8$ , and  $\rho = 1/2$ . Evaluating  $f$  on  $D = \{3, 10, 5, 11, 14, 7, 12, 6\}$  gives the codeword

$$(f(x))_{x \in D} = (6, 3, 8, 15, 16, 4, 8, 16).$$

Splitting  $f$  as in Definition 3.2 gives  $f_e(Y) = 1 + 3Y$  and  $f_o(Y) = 2 + 4Y$ . With the challenge  $\alpha = 5$ , the fold is

$$f^{(5)}(Y) = (1 + 3Y) + 5(2 + 4Y) = 11 + 6Y,$$

of degree  $1 < 2$ , exactly as Definition 3.3 predicts. Applying Equation (3.9) to the four pairs  $(f(x), f(-x))$  gives the codeword  $(14, 16, 8, 6)$  on  $D^{(2)} = \{9, 15, 8, 2\}$ , which is precisely the evaluation of  $11 + 6Y$  on that domain. Both the degree bound and the domain size have halved, and the rate is still  $2/4 = 1/2$ .

## 3.4 The Protocol

The folding operation of Section 3.3 is the engine; the protocol is what wraps it into an interaction between prover and verifier. It has two phases. In the *commit phase* the prover folds the polynomial down to a short final layer, committing to each intermediate codeword with a Merkle tree so that it cannot change its mind later. In the *query phase* the verifier picks a few random positions and asks the prover to reveal the values needed to re-check that each fold was done correctly. If the prover cheated, that is, if the starting codeword was not close to a low-degree polynomial, then at least one of these checks fails with high probability.

Throughout,  $D_0 = D$  is the initial domain of size  $N$ , and  $D_r$  is the domain after  $r$  folds, of size  $N/2^r$ .

### 3.4.1 Commit Phase

The prover starts from the coefficients of  $f$ , extends them to a codeword on  $D$  by the low-degree extension (Subsection 1.2.2), and then repeats the same three actions at every round: commit, receive a challenge, fold.

1. **Commit.** Send the oracle for the current codeword. This binds the prover to that exact codeword: every later query must be answered consistently with it.
2. **Challenge.** Obtain the folding challenge  $\alpha_r$  from the verifier (once made non-interactive by the BCS transformation of Section 2.3, this challenge is derived from the transcript).
3. **Fold.** Apply Equation (3.9) with  $\alpha_r$  to halve the codeword, and square the two domain parameters (Equation (3.4)) to move to  $D_{r+1}$ .

The process stops once the codeword has shrunk to a short final layer, which the prover sends directly. The number of rounds is  $\log_2 N$  down to a constant, or  $\log_2(N/\ell)$  if the prover stops at a final layer of length  $\ell$ .

In the IOP the prover sends, at each round, the oracle for the current codeword, and the verifier replies with the folding challenge; the concrete realisation of these oracles as commitments follows the BCS transformation of Section 2.3. For a polynomial of length 8 with blowup 2, so  $N = 16$ , the process runs  $\log_2 N = 3$  rounds down to a constant final layer.

### 3.4.2 Query Phase

Committing to the codewords is not enough on its own: the verifier must check that consecutive layers are actually related by the fold. It does so by sampling a small number of random positions and, at each one, asking the prover for just the values needed to recompute one fold.

For a single query the verifier picks an index and follows it down through the layers. At round  $r$ , with domain size  $n_r$ , the index determines a pair of positions half the list apart,

$$\text{lo} = \text{idx} \bmod (n_r/2), \quad \text{hi} = \text{lo} + n_r/2,$$

which are exactly the entries  $f(x)$  and  $f(-x)$  that Equation (3.9) combines. The prover returns both values together with their Merkle authentication paths. The index for the next round is then  $\text{idx} \leftarrow \text{lo}$ , so one random choice in the first layer determines the whole chain of positions down to the final layer.

In IOP terms, the verifier samples a number of indices in  $[0, N/2)$  and, for each one, queries the two oracle positions per layer needed to recompute one fold. Each query thus costs one pair of oracle queries per round.

### 3.4.3 Verification and the Colinearity Check

The verifier holds only the proof, namely the Merkle roots, the final layer, and the opened values with their paths, and must decide whether to accept. It proceeds in three steps.

First, it **reconstructs the challenges**. Re-reading the roots and the final layer from the transcript in the same order the prover wrote them, it recomputes every  $\alpha_r$  and every domain parameter, and checks that the number of rounds matches the expected  $\log_2(N/\ell)$ .

Second, for each queried position it **checks the Merkle paths**, confirming that the two opened values really sit at their claimed positions under the committed root. A failed path means the prover is opening something it did not commit to.

Third, it performs the **colinearity check**. Using the opened pair  $(\text{lo}, \text{hi})$  at round  $r$ , it recomputes the folded value by the same rule the prover should have used,

$$v = \frac{\text{lo} + \text{hi}}{2} + \alpha_r \cdot \frac{\text{lo} - \text{hi}}{2x}, \quad x = \text{offset}_r \cdot \omega_r^{\text{lo}}, \quad (3.10)$$

and compares  $v$  against the value at the corresponding position of the *next* layer, which is an opened entry for intermediate rounds, or the final layer for the last round. If the two disagree, the fold was not performed honestly and the verifier rejects.

The name *colinearity check* comes from the geometry of Equation (3.10). The two points  $(x, \text{lo})$  and  $(-x, \text{hi})$ , together with the folded value at  $\alpha_r$ , must lie on a single line, since the line through  $f(x)$  and  $f(-x)$  evaluated at  $\alpha_r$  is exactly  $f^{(\alpha_r)}(x^2)$ . Each query verifies one such local relation per layer; a codeword far from low degree cannot satisfy all of them at once, so a handful of random queries suffices to catch a cheating prover.

In IOP terms the verifier reconstructs the challenges from the transcript, checks that each queried oracle answer is consistent (which, after compilation, means checking a

commitment opening), and verifies the colinearity relation Equation (3.10) at every queried position of every layer.

## 3.5 Soundness

The earlier sections describe an honest prover. Soundness asks the opposite: if the committed function is *not* close to any low-degree polynomial, can a cheating prover still make the verifier accept? We show the answer is no, except with small probability, following the round-by-round analysis of Garreta, Mohnblatt, and Wagner [GMW25], which gives the simplest known soundness proof for FRI in the form implemented here.

One point frames everything that follows. FRI certifies *proximity*, not equality (Remark 3.1): the verifier never learns that the committed vector is exactly a codeword, only that it is close to one. Soundness must therefore be stated in terms of distance. We fix a distance parameter  $\delta$  and ask that, if the initial function is  $\delta$ -far from the code  $\text{RS}[\mathbb{F}, D, \rho]$  in relative Hamming distance (Definition 1.8), then the verifier rejects with high probability. The difficulty is that the protocol only ever inspects a few points, so we must argue that being far from the code cannot quietly disappear during folding, and that it is caught by the queries.

### 3.5.1 The Two Things That Can Go Wrong

A cheating prover has exactly two ways to succeed, and the proof bounds each in turn.

The first is that *folding rescues a far function*. Each round replaces the current layer  $f_i$  by  $f_{i+1} = f_{i,e} + \alpha_i f_{i,o}$ , a random linear combination of its two halves. If some challenge  $\alpha_i$  made  $f_{i+1}$  close to the code even though  $f_i$  was far, the prover would have folded its way from a dishonest layer to an honest-looking one, and no amount of querying would detect it. So we need folding to *preserve distance*: for all but a few unlucky  $\alpha_i$ , a far layer stays far.

The second is that *the queries all miss*. Even if every layer stays far, the verifier only checks a few random positions. If the far parts of the final layer happened to avoid every queried position, the cheat would slip through. So we need enough queries that missing all of them is unlikely.

The next two subsections address these in order: first the algebraic fact that makes folding preserve distance, then the theorem that combines it with the query bound.

### 3.5.2 Why Folding Preserves Distance

Distance preservation is not obvious, precisely because  $f_{i+1}$  is a chosen linear combination of  $f_{i,e}$  and  $f_{i,o}$ , and one might fear a lucky  $\alpha_i$  that makes it close to the code by accident. The property that rules this out is *correlated agreement*, introduced in the study of proximity gaps [BS+20]: if a random combination of several functions is close to the code, then those functions are not just individually close, they agree with codewords on a *common* set of positions.

The round-by-round proof uses a sharper version, *mutual* correlated agreement, introduced in the analysis of WHIR [Arn+24]. It requires that the set of positions where the combination agrees with the code is the *same* set on which the two halves

agree, not merely a related one. This equality of sets is what lets the argument repeat cleanly at every round: the agreement pattern of one layer passes to the next without drift, so distance is tracked faithfully all the way down. This is why we use mutual correlated agreement rather than the older notion.

Crucially, mutual correlated agreement is a *theorem*, not an assumption, as long as the distance stays below the Johnson radius  $1 - \sqrt{\rho}$  (Theorem 1.1) [GMW25]. This is the regime in which the soundness of our implementation is established, and we return to what lies beyond it at the end of the section.

### 3.5.3 The Soundness Theorem

With distance preservation in hand, the two risks of Subsection 3.5.1 become two error terms.

**Theorem 3.2** (Round-by-round soundness of FRI, informal [GMW25, Theorem 5.2]). *Let  $f$  be  $\delta$ -far from  $\text{RS}[\mathbb{F}, D, \rho]$  with  $\delta$  below the Johnson radius  $1 - \sqrt{\rho}$ . Then any cheating prover makes the verifier accept with probability at most*

$$\underbrace{\frac{r}{|\mathbb{K}|} \cdot \text{poly}(|D|)}_{\text{folding}} + \underbrace{(1 - \delta)^s}_{\text{queries}},$$

where  $\mathbb{K}$  is the field the challenges are drawn from,  $r = \log_2(|D|/\ell)$  the number of rounds, and  $s$  the number of queries.

*Proof sketch.* The proof tracks one bit of information through the protocol: whether the current layer is still far from its code. Following [GMW25] we call the state *doomed* while this holds. The argument is that a doomed state stays doomed through folding, and that a doomed final state is caught by the queries.

*The start is doomed.* By assumption  $f_0$  is  $\delta$ -far from the code, so the state is doomed before any interaction. The prover has committed to a far vector and cannot take it back.

*Folding keeps it doomed* ([GMW25, Lemma 5.1]). Suppose layer  $f_i$  is  $\delta$ -far. Its fold is  $f_{i+1} = f_{i,e} + \alpha_i f_{i,o}$ . If  $f_{i+1}$  were close to the code for many values of  $\alpha_i$ , mutual correlated agreement (Subsection 3.5.2) would force  $f_{i,e}$  and  $f_{i,o}$  to agree with codewords on a common set large enough to make  $f_i$  itself close, which contradicts the assumption. Hence only a few  $\alpha_i$  are bad, a fraction of about  $1/|\mathbb{K}|$  of the field. A union bound over the  $r$  rounds gives the folding term. This is the only place the field size enters, and it is why  $\mathbb{K}$  must be the large extension  $\mathbb{F}_{p^4}$ : a challenge from the base field would make  $1/|\mathbb{K}|$  far too large.

*The queries catch a doomed final layer* ([GMW25, Claim 5.3]). If every fold kept the state doomed, the final layer sent in the clear is itself  $\delta$ -far from a low-degree polynomial. A single query picks a random position and checks the colinearity relation Equation (3.10) along the chain; since the final layer disagrees with the code on at least a  $\delta$  fraction of positions, the query exposes a broken fold with probability at least  $\delta$ . The  $s$  queries are independent, so all of them miss with probability at most  $(1 - \delta)^s$ , the query term.

The prover wins only if some fold got lucky or every query missed, so the total is the sum of the two terms.  $\square$

The two terms are controlled by different means, which is what makes the protocol practical. The folding term is negligible for free: with  $\mathbb{K} = \mathbb{F}_{p^4}$  of size about  $2^{124}$  (Remark 3.2) and only a handful of rounds, it sits around  $2^{-122}$ . The query term is the one the designer tunes, by choosing how many queries to make.

**Example 3.3.** Take a final layer that disagrees with the code on half its positions,  $\delta = 1/2$ , and  $s = 20$  queries. One query misses with probability  $1/2$ , so twenty independent queries all miss with probability  $(1/2)^{20} \approx 10^{-6}$ , about 20 bits of security from the query phase. A smaller rate raises the detection probability per query and so buys more bits each. The concrete parameters used in this work, and the resulting security level, are discussed in ??.

### 3.5.4 Beyond the Provable Regime

Everything above lives below the Johnson radius  $1 - \sqrt{\rho}$ , where mutual correlated agreement is proved. One can push further, up to the list-decoding *capacity*  $1 - \rho$ , by *conjecturing* that the same distance preservation still holds; this would allow fewer queries for the same security. That regime is attractive for efficiency but unproven, and recent work has cast doubt on the strongest such conjectures. We therefore rely only on the provable Johnson-radius guarantee and leave the capacity regime out of scope.



# Bibliography

- [Arn+24] Gal Arnon et al. *WHIR: Reed-Solomon Proximity Testing with Super-Fast Verification*. Cryptology ePrint Archive, Paper 2024/1586. <https://eprint.iacr.org/2024/1586>. 2024.
- [BS+18] Eli Ben-Sasson et al. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. Cryptology ePrint Archive, Report 2018/046. <https://eprint.iacr.org/2018/046>. 2018.
- [BS+20] Eli Ben-Sasson et al. “Proximity Gaps for Reed-Solomon Codes”. In: *Proceedings of the 61st IEEE Symposium on Foundations of Computer Science (FOCS)*. 2020.
- [BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. “Interactive Oracle Proofs”. In: *Theory of Cryptography Conference (TCC)*. IACR ePrint 2016/116. <https://eprint.iacr.org/2016/116>. 2016.
- [CO25] Alessandro Chiesa and Eylon Orrù. *Fiat-Shamir via Sponges*. Cryptology ePrint Archive, Report 2025/536. <https://eprint.iacr.org/2025/536>. 2025.
- [GMW25] Albert Garreta, Nicolas Mohnblatt, and Benedikt Wagner. *A Simplified Round-by-round Soundness Proof of FRI*. Cryptology ePrint Archive, Paper 2025/1993. <https://eprint.iacr.org/2025/1993>. 2025.
- [Sta21] StarkWare. *ethSTARK Documentation*. Cryptology ePrint Archive, Report 2021/582. <https://eprint.iacr.org/2021/582>. 2021.
- [Tha22] Justin Thaler. *Proofs, Arguments, and Zero-Knowledge*. <https://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.html>. Foundations, Trends in Privacy, and Security, 2022.